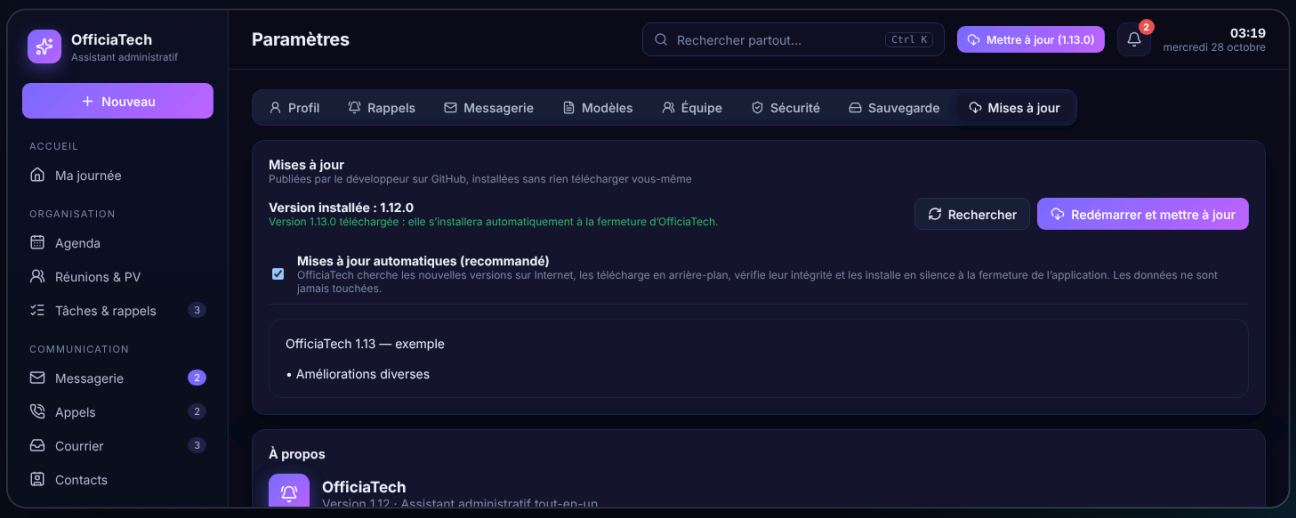
OfficiaTech
Version 1.12 · document interne

Guide du propriétaire et du développeur

Publier une mise à jour, installer un nouvel ordinateur, dépanner la messagerie, entretenir le serveur, et tout ce qu'un développeur doit savoir pour faire évoluer OfficiaTech sans rien casser.

Mises à jour automatiques · GitHub Actions · Édition générale



M. TETCHI KOREY JOEL, Comptable — propriétaire et développeur
+225 07 19 37 80 57 · +225 01 60 29 61 49 · heliontech97@gmail.com

Édition d'octobre 2026

© 2026 M. TETCHI KOREY JOEL

Sommaire

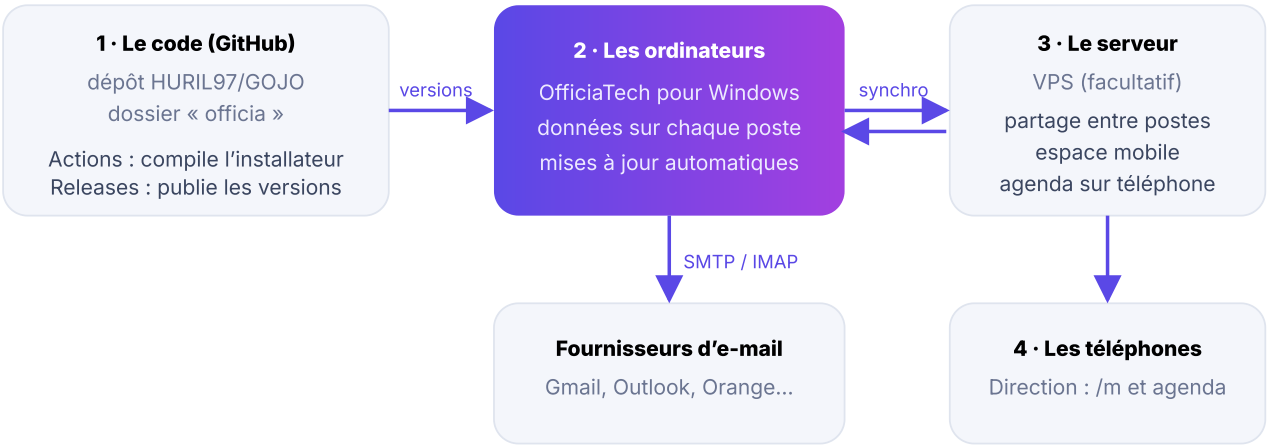
1	Vue d’ensemble	les pièces du système et l’historique des versions
2	Où se trouve quoi	fiche à garder
3	Publier une mise à jour	pas à pas, en 10 minutes
4	Ce que voient les utilisateurs	mise à jour automatique
5	Installer un nouvel ordinateur	première installation
6	Messagerie : diagnostic et solutions	antivirus, ports, mots de passe
7	Le serveur d’équipe	rappel et mise à jour
8	Pour le développeur : le projet	architecture, dossiers, commandes
9	Pour le développeur : règles d’or	données, versions, sécurité
10	Pour le développeur : mécanique des mises à jour	GitHub, installateur, secours
11	Incidents et retour arrière	que faire si une version pose problème
A	Aide-mémoire	checklists et commandes

Les chapitres marqués **Propriétaire** se lisent sans connaissances techniques ; ceux marqués **Développeur** s’adressent à la personne qui modifie le code.

1 Vue d’ensemble

Propriétaire

OfficiaTech repose sur quatre pièces. Seule la première est indispensable ; les autres apportent les mises à jour automatiques, le travail en équipe et l’accès mobile.



Historique des versions

Version	Contenu principal
1.00	Première version : agenda, réunions et PV, messagerie, courrier, appels, tâches, rappels, notifications, sessions.
1.10	Connexion Microsoft, dossiers Envoyés, réponses rapides, invitations calendrier, rappel des visiteurs, feuille de présence, suivi des actions, numérisation et circuit du courrier, lettres à en-tête, verrouillage, sauvegarde externe, serveur d’équipe et espace mobile, rapport d’activité.
1.11	Correctif messagerie : envoi sur les postes avec un antivirus qui analyse les e-mails ; diagnostic détaillé et port de secours.
1.12	**Mises à jour automatiques** : recherche, téléchargement vérifié et installation silencieuse.

À retenir

À partir de la version 1.12, vous n’avez plus jamais à distribuer l’installateur : il suffit de **publier** une nouvelle version sur GitHub (chapitre 3) ; chaque ordinateur la trouve, la télécharge et l’installe tout seul.

2 Où se trouve quoi

Propriétaire

Fiche de référence. Version générale (sans les accès du serveur).

GitHub	
Dépôt du code	https://github.com/HURIL97/GOJO (dossier `officia`)
Versions publiées	https://github.com/HURIL97/GOJO/releases (étiquettes `officia-vX.Y.Z`)
Compilation	onglet **Actions** › « OfficiaTech — installateur Windows »
Notes de version	`officia/CHANGELOG.md` (affichées dans l'application)
Numéro de version	`officia/package.json`, ligne `"version"`
Manuels (PDF)	`officia/docs/`
Sur chaque ordinateur (Windows)	
Programme	`%LOCALAPPDATA%\Programs\OfficiaTech` (installation pour l'utilisateur)
Données	`%APPDATA%\OfficiaTech\data` : sessions (`profils`), sauvegardes (`sauvegardes`), pièces jointes (`fichiers`)
Mises à jour téléchargées	`%APPDATA%\OfficiaTech\mises-a-jour` (vidé automatiquement)
Sauvegarde externe	dossier choisi dans Paramètres › Sauvegarde (clé USB, Google Drive, OneDrive…)
Serveur d'équipe (VPS)	
Détails	voir la fiche du « Guide du serveur d'équipe » (adresse, accès SSH, dossiers)

Ce qu'il ne faut jamais perdre

L'accès au compte GitHub **HURIL97** (sans lui, plus de mises à jour), la **clé SSH du VPS**, le **jeton de l'équipe**, et une sauvegarde récente des données de chaque poste.

3 Publier une mise à jour

Propriétaire Développeur

Une fois les modifications du code enregistrées sur GitHub, publier une version prend une dizaine de minutes, dont cinq d'attente.



① Changer le numéro de version

Dans `officia/package.json`, augmentez la ligne `"version"`. Le numéro a trois parties **MAJEUR.MINEUR.CORRECTIF** :

Type de changement	Exemple
Correction de bug	1.12.0 → **1.12.1**
Nouvelles fonctions	1.12.1 → **1.13.0**
Refonte importante	1.13.0 → **2.0.0**

Mettez aussi à jour l'affichage « À propos » : `officia/src/lib/about.js`, ligne `version: '1.13'`.

> **Règle absolue**
> Le numéro doit toujours **augmenter** et ne jamais resservir. Les ordinateurs n'installent qu'une version **plus grande** que la leur : publier 1.12.0 une deuxième fois, ou 1.11.9 après 1.12.0, ne fait rien.

② Décrire les nouveautés

Ajoutez en haut de `officia/CHANGELOG.md` un court texte en français, pour les utilisateurs : il s'affiche dans l'application avant l'installation et sur la page de la version.

```
OfficiaTech 1.13 — titre court

• Ce qui change pour l'utilisateur, en une phrase.
• Une autre nouveauté ou correction.
```

③ Vérifier et envoyer sur GitHub

```
$ cd officia
$ npm test          # tous les tests doivent passer (« fail 0 »)
$ npm run build     # vérifie que l'interface se construit
$ git add -A && git commit -m "OfficiaTech 1.13 : …" && git push origin main
```

Chaque envoi sur **main** lance déjà la compilation (tests + installateur), **sans publier** : les ordinateurs ne reçoivent rien tant que l'étape ④ n'est pas faite.

④ Publier

1. Sur https://github.com/HURIL97/GOJO, ouvrez l'onglet **Actions**.
2. Dans la liste de gauche, choisissez **« OfficiaTech — installateur Windows »**.
3. À droite, bouton **Run workflow** : laissez la branche **main**, **cochez « publier »**, puis le bouton vert **Run workflow**.
4. Attendez environ 5 minutes que la ligne devienne verte ✓ (tests, puis compilation Windows, puis publication).
5. Vérifiez dans **Releases** qu'une version **officia-v1.13.0** est apparue avec `OfficiaTech-Setup-1.13.0.exe`.

> **Autre méthode (développeur)**
>
> Pousser une étiquette publie aussi la version : `git tag officia-v1.13.0 && git push origin officia-v1.13.0`. Le numéro de l'étiquette doit être celui de `package.json`.

⑤ C'est tout

Chaque ordinateur vérifie au démarrage puis toutes les 4 heures. Il télécharge la version en arrière-plan, puis l'installe à la fermeture d'OfficiaTech (ou immédiatement si l'utilisateur clique sur « Mettre à jour »). Un ordinateur éteint la recevra à son prochain démarrage.

4 Ce que voient les utilisateurs

Propriétaire

La mise à jour est discrète : rien n'interrompt le travail en cours.

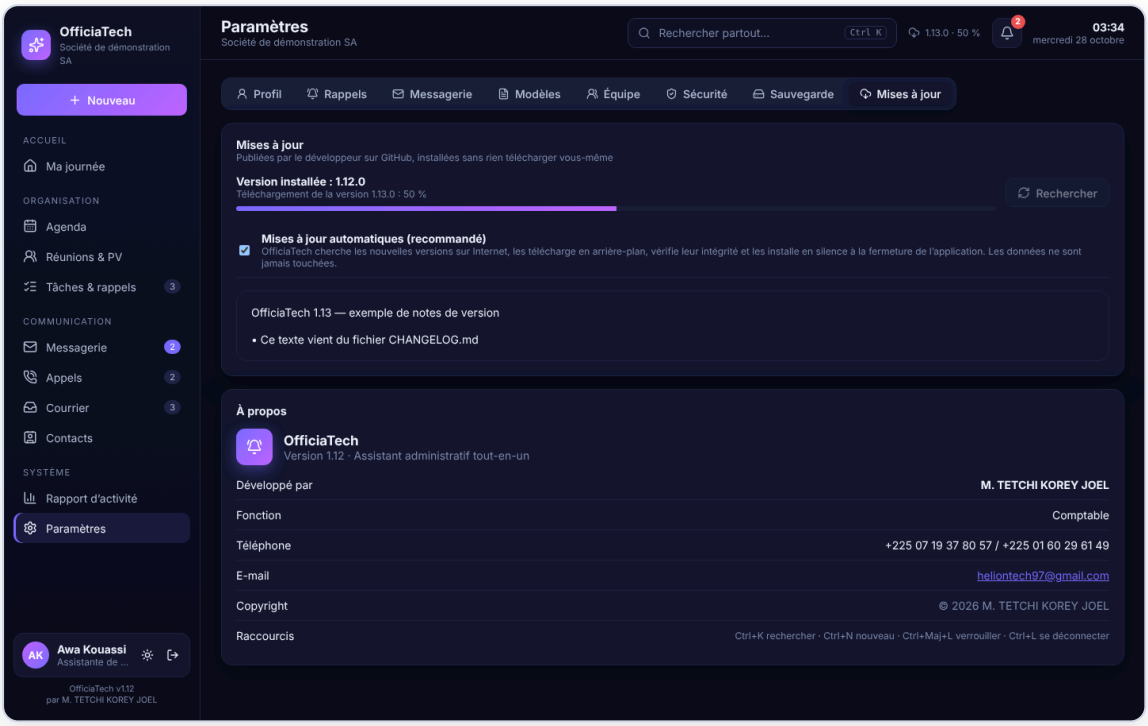


Figure 4.1 — Téléchargement en arrière-plan : pourcentage en haut de l'écran et dans Paramètres › Mises à jour

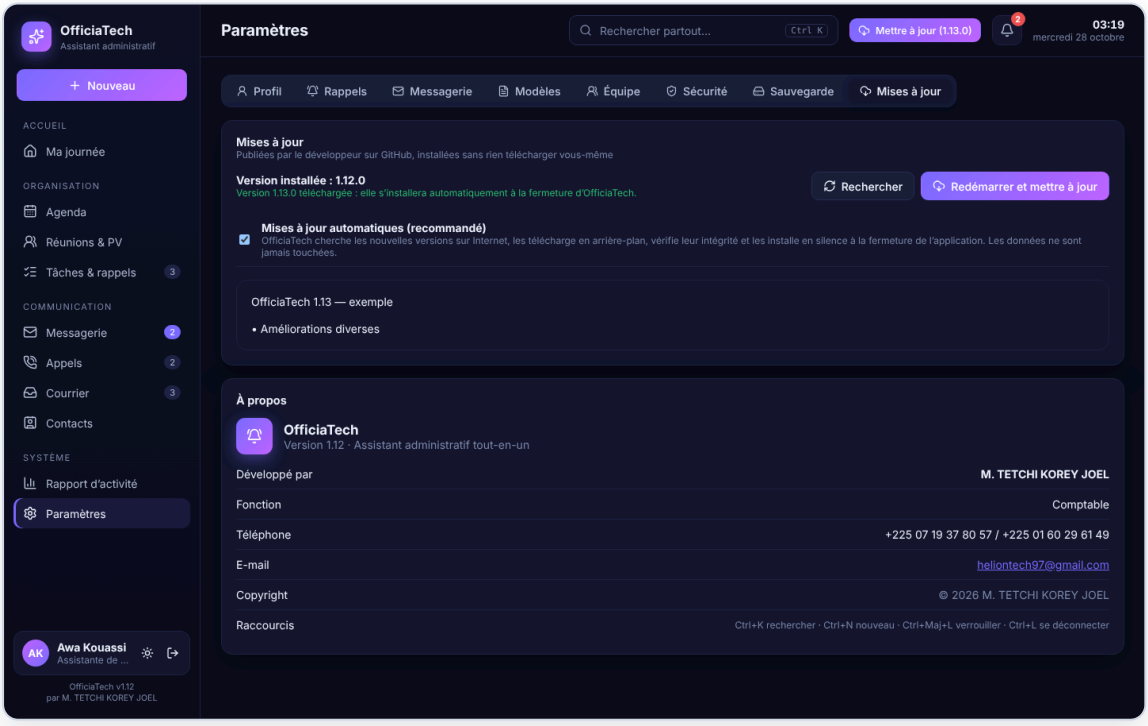


Figure 4.2 — Version téléchargée et vérifiée : bouton « Mettre à jour » en haut de l'écran, notes de version

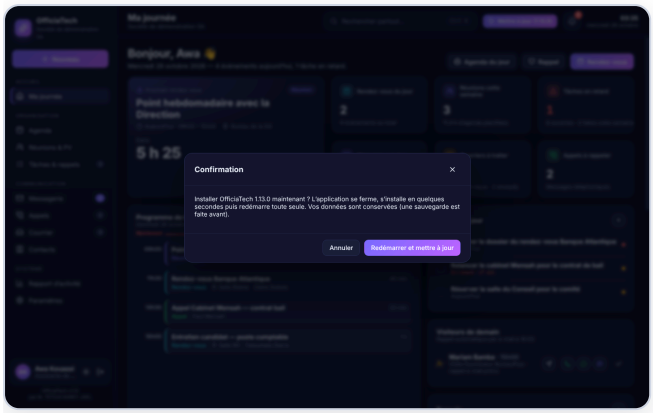


Figure 4.3 — Confirmation avant l'installation immédiate

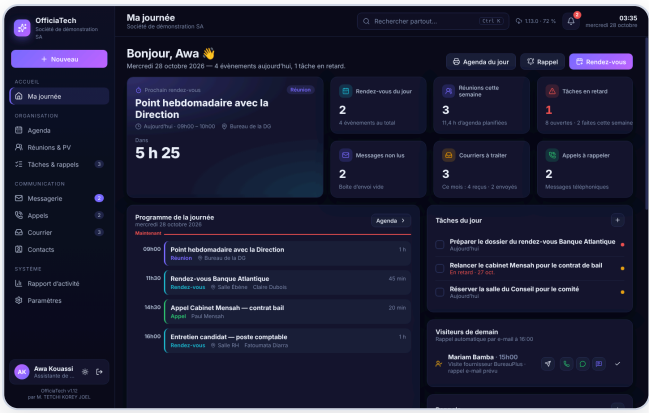


Figure 4.4 — Pendant le téléchargement, on continue de travailler

Situation	Ce qui se passe
L'utilisateur clique sur « Mettre à jour »	Sauvegarde des données, fermeture, installation silencieuse (quelques secondes), OfficiaTech **redémarre tout seul**.
L'utilisateur ne fait rien	La version s'installe **à la fermeture** d'OfficiaTech ; au prochain lancement, la nouvelle version est là.
Pas d'Internet	Rien ne se passe ; nouvel essai 4 heures plus tard ou au prochain démarrage.
Mises à jour automatiques désactivées	OfficiaTech signale la nouvelle version ; l'utilisateur la télécharge d'un clic dans Paramètres › Mises à jour.

Sécurité du téléchargement

Seuls les fichiers venant de la page des versions du dépôt HURIL97/GOJO sont acceptés. Leur taille et leur empreinte SHA-256 (fournie par GitHub) sont contrôlées : un fichier incomplet ou modifié est rejeté et n'est jamais lancé.

5 Installer un nouvel ordinateur Propriétaire

La seule fois où l'on télécharge l'installateur à la main.

1. Sur l'ordinateur, ouvrez https://github.com/HURIL97/GOJO/releases et téléchargez **OfficiaTech-Setup-x.y.z.exe** de la version la plus récente.
2. Lancez-le. Si Windows affiche « Windows a protégé votre ordinateur » : **Informations complémentaires** › **Exécuter quand même** (l'application n'est pas encore signée numériquement).
3. Gardez l'installation **pour l'utilisateur courant** (choix par défaut) : les mises à jour se feront sans demander de droits administrateur.
4. Au premier lancement, créez la session de données réelles (et, si besoin, une session de démonstration).
5. Configurez : **messagerie** (Paramètres › Messagerie, chapitre 6), **équipe** (Paramètres › Équipe, chapitre 7), **sauvegarde externe** (Paramètres › Sauvegarde), **code de session** (Paramètres › Sécurité).

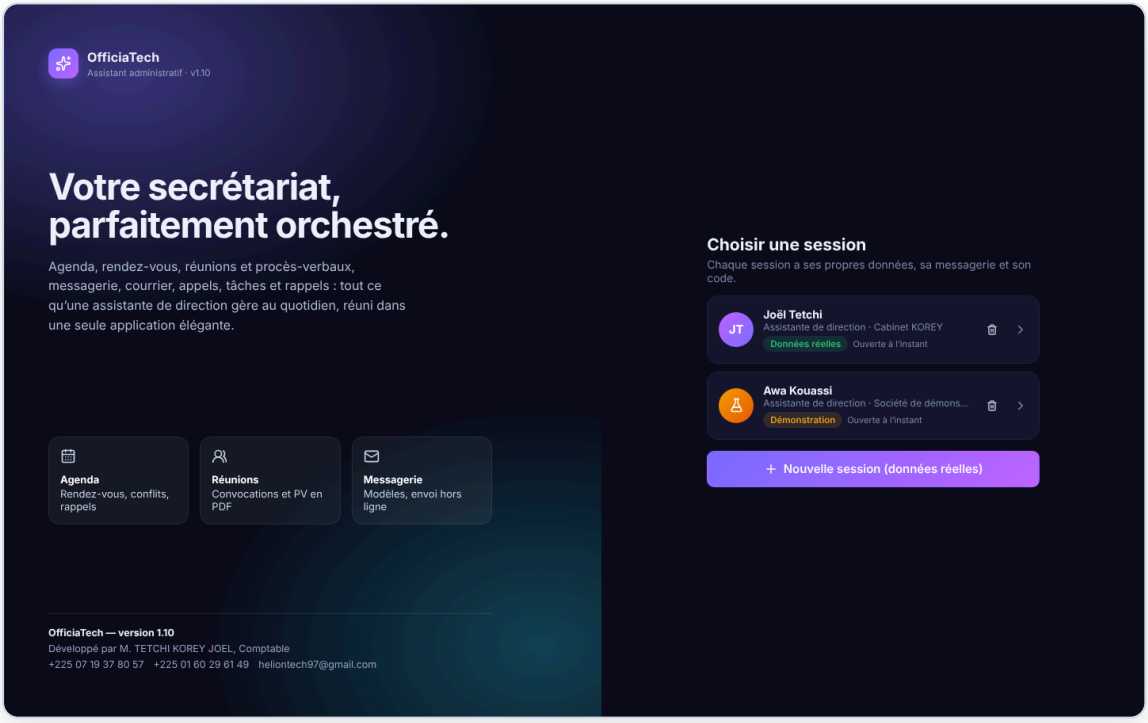


Figure 5.1 — Premier lancement : choix ou création de la session

Ordinateurs encore en 1.00, 1.10 ou 1.11

L'automatisme existe depuis la 1.12. En **1.10 / 1.11** : Paramètres › Mises à jour › **Installer** (l'assistant s'ouvre une dernière fois). En **1.00** : installer la dernière version comme un nouvel ordinateur ; les données sont conservées.

6 Messagerie : diagnostic et solutions

Propriétaire

Quand une adresse fonctionne sur un ordinateur et pas sur un autre, la cause est presque toujours **l'ordinateur** (antivirus, réseau), pas le compte.

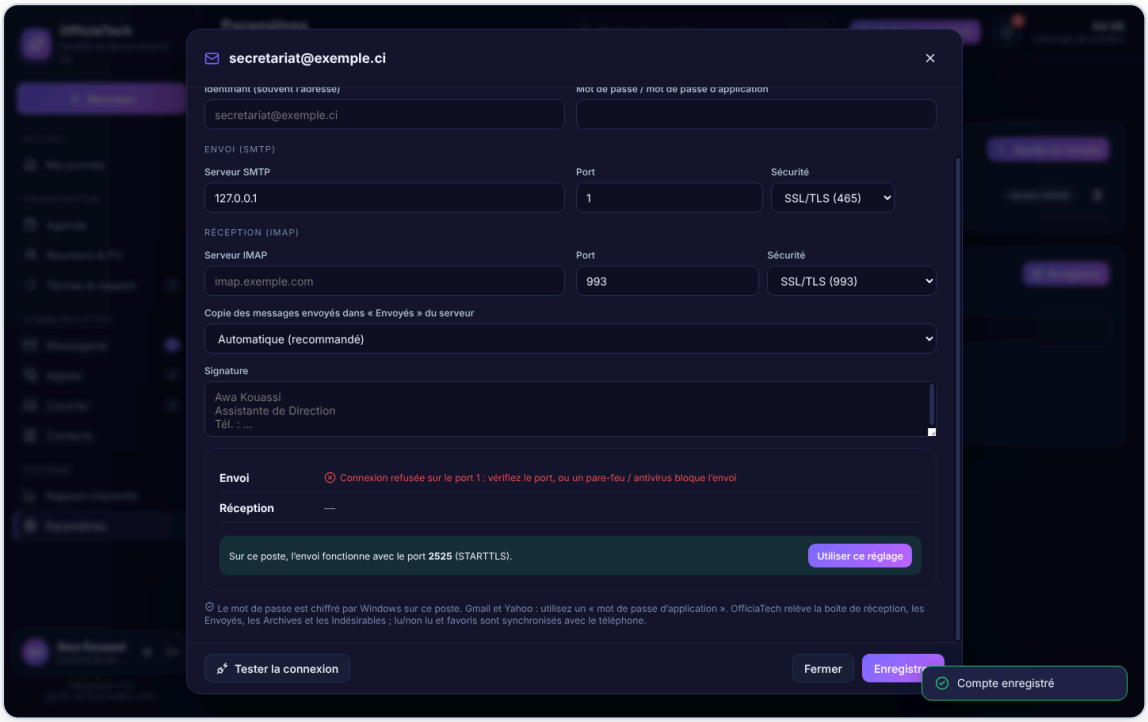


Figure 6.1 — « Tester la connexion » : envoi bloqué sur ce poste, un autre port fonctionne

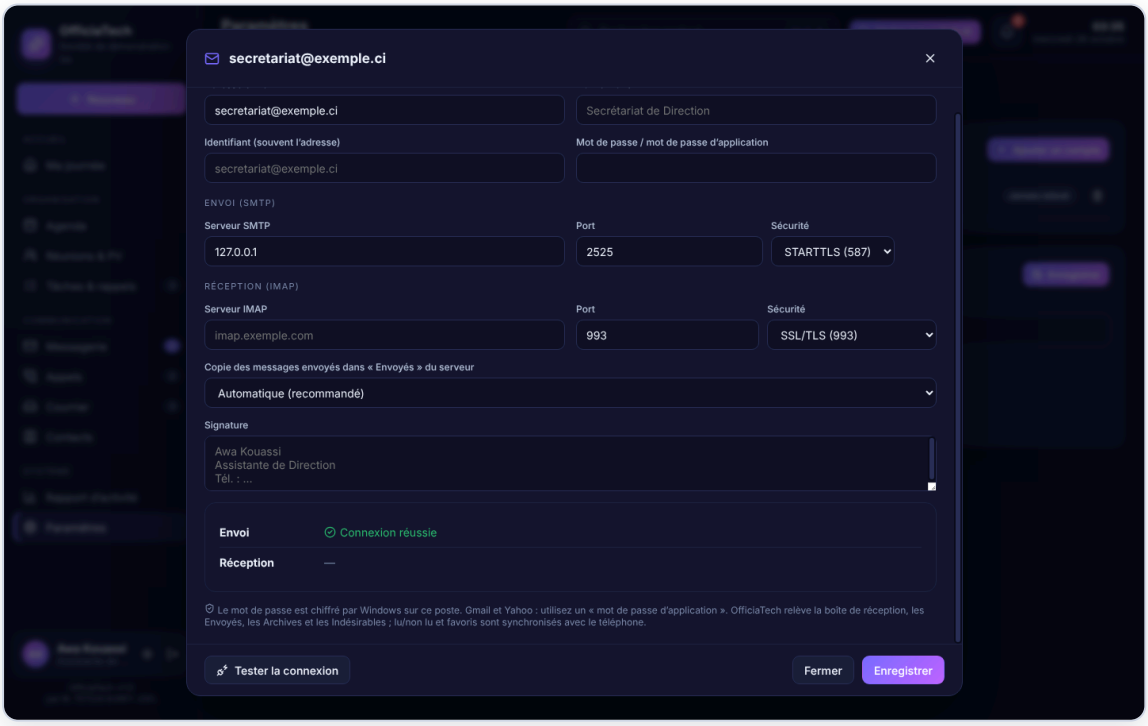


Figure 6.2 — Après « Utiliser ce réglage » : connexion réussie

Message affiché	Cause et solution
Certificat de sécurité non reconnu	Un **antivirus** (Avast, AVG, Kaspersky, ESET…) analyse les e-mails. Depuis la 1.11, OfficiaTech accepte les certificats installés dans Windows ; si le message persiste, désactivez dans l'antivirus l'« analyse des e-mails sortants (SMTP) » ou ajoutez OfficiaTech en exception.
Connexion refusée / ne répond pas sur le port …	Port bloqué par le réseau ou le pare-feu. OfficiaTech essaie seul les ports 587, 465 et 2525 et propose celui qui marche : cliquez sur **Utiliser ce réglage**.
Mauvais réglage de sécurité pour ce port	465 = SSL/TLS ; 587 = STARTTLS.
Identifiant ou mot de passe refusé	Gmail, Yahoo : **mot de passe d'application** (et non le mot de passe habituel).
Connexion refusée par Microsoft	Outlook, Hotmail, Microsoft 365 : bouton **Se connecter avec Microsoft**, après l'enregistrement unique de l'application sur entra.microsoft.com (étapes affichées dans Paramètres › Messagerie).
Serveur introuvable	Adresse du serveur mal saisie, ou pas d'Internet.

7 Le serveur d’équipe Propriétaire

Le serveur du VPS a son propre guide : **« Guide du serveur d’équipe et de la connexion »**. L’essentiel à retenir :

- Les mises à jour automatiques concernent **l’application Windows**. Le **serveur** se met à jour à part, en relançant la commande d’installation depuis PowerShell (jeton, données et adresse sont conservés) :

```
PS> ssh -i C:\chemin\vers\cle root@IP_DU_VPS "curl -fsSL https://raw.githubusercontent.com/HURIL97/GOJO/main/officia/server/install-vps.sh | bash"
```

- Ne le mettre à jour que si une version modifie le dossier `officia/server` (c’est indiqué dans le CHANGELOG).
- Sur chaque poste : Paramètres › Équipe, adresse **https://equipe.votre-domaine.com** et le jeton de l’équipe.
- État du service : `systemctl status officiatech` ; journal : `journalctl -u officiatech -f`.

8 Pour le développeur : le projet

Développeur

Application Electron (Windows), interface React, base SQLite intégrée à Node.js : aucun module natif à compiler.

Fichier ou dossier	Rôle
`electron/main.js`	Processus principal : fenêtre, icône près de l'horloge, notifications, sessions, verrouillage, IPC (`handle('canal', fn)`), PDF, sauvegardes, planificateur (30 s), automatisations
`electron/db.js`	Base SQLite (`node:sqlite`) ; schéma dans `TABLES` , migrations automatiques, colonnes de synchronisation
`electron/services.js`	Métier : agenda et récurrences, réunions et PV, tâches, rappels, notifications, courrier, appels, recherche, tableau de bord, rapport mensuel, paramètres par défaut (`DEFAULT_SETTINGS`)
`electron/mail.js`	Messagerie : SMTP (nodemailer), IMAP (imapflow), dossiers, drapeaux, boîte d'envoi, diagnostic
`electron/oauth.js`	Connexion Microsoft (OAuth 2.0 + PKCE)
`electron/ics.js`	Calendriers .ics (invitations, import/export, flux) — utilisé aussi par le serveur
`electron/sync.js`	Synchronisation avec le serveur d'équipe
`electron/updater.js`	Mises à jour automatiques
`electron/profiles.js`	Sessions (une base par session)
`src/`	Interface React : `pages/` (écrans), `components/` (fenêtres), `lib/print.js` (documents PDF), `lib/about.js` (version affichée)
`server/`	Serveur d'équipe (Node.js sans dépendance), page mobile, script `install-vps.sh`
`tests/`	25 tests automatiques (agenda, PV, courrier, messagerie avec vrais serveurs SMTP/IMAP, OAuth, synchronisation, mobile, mises à jour)
`.github/workflows/officia-windows.yml`	Compilation et publication (à la racine du dépôt)

Commandes

```
$ cd officia
$ npm install            # une fois (Node.js 24)
$ npm run dev            # interface + Electron avec rechargement
$ npm test               # 25 tests
$ npm run build          # construit l'interface (dist/)
$ npm run dist:win       # installateur Windows (sur un PC Windows), dans release/
```

Ajouter une fonction : le chemin habituel

1. **Donnée** : ajouter la colonne dans `TABLES` (`db.js`) — elle est créée automatiquement au démarrage sur les bases existantes.
2. **Métier** : écrire la fonction dans `services.js` (ou `mail.js`), avec un test dans `tests/`.
3. **Canal** : l'exposer dans `main.js` avec `handle('module.action', …)`.
4. **Écran** : l'appeler depuis React avec `call('module.action', …)`.
5. **Réglage** éventuel : l'ajouter à `DEFAULT_SETTINGS` (sinon il n'est pas enregistré).

Compilation sur GitHub

Le workflow a deux étapes : **test** (Linux : `npm ci`, `npm test`, `npm run build`) puis **windows** (installateur NSIS et version portable, publiés en artefact, et en version quand « publier » est coché ou qu'une étiquette `officia-v*` est poussée). Si les tests échouent, rien n'est compilé ni publié.

9 Pour le développeur : règles d'or Développeur

Les erreurs à ne jamais commettre : elles toucheraient tous les ordinateurs à la mise à jour suivante.

Règle	Pourquoi
Ne jamais renommer ni supprimer une table ou une colonne	Les bases existantes des utilisateurs seraient incompatibles. On **ajoute** seulement (les migrations ajoutent les colonnes manquantes).
Ne pas changer le format d'une valeur enregistrée	Les dates sont des textes locaux `AAAA-MM-JJTHH:MM:SS` ; les listes sont du JSON. Les anciennes données doivent rester lisibles.
Ne pas changer `appId` (`com.officiatech.app`) ni le nom du produit	Windows verrait une autre application : installation en double, données « perdues », mises à jour cassées.
Garder le nom des fichiers publiés	`OfficiaTech-Setup-${version}.exe` : la mise à jour (et sa source de secours) le recherche ainsi.
Toujours augmenter la version	Voir chapitre 3.
Tables partagées : liste `SYNCED` (`db.js`) et `TABLES` du serveur	Pour partager une nouvelle table entre postes, l'ajouter aux deux, puis mettre à jour le serveur.
Ne jamais publier de secret	Le dépôt est **public** : aucun mot de passe, jeton, clé, adresse IP de serveur ni donnée client dans le code ou les documents.
Tester avant de publier	`npm test` sans échec, et essai sur un poste avant la publication.

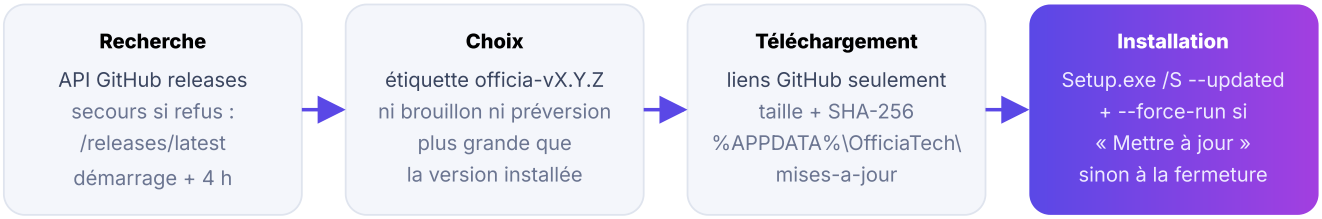
Variables utiles pour les essais

`OFFICIA_DEV=1` (interface en mode développement) · `OFFICIA_DEBUG=1` (détail des erreurs de messagerie) · `OFFICIA_UPDATE_FEED=http://…` (simuler une publication sans toucher à GitHub).

10 Pour le développeur : mécanique des mises à jour

Développeur

Module `electron/updater.js`, testé par `tests/updater.test.js`.



- **États** envoyés à l'interface (`update:state`) : idle, checking, uptodate, available, downloading (pourcentage), ready, installing, error.
- **Réglage** automatique / manuel : fichier `update.json` du dossier des mises à jour (commun à toutes les sessions).
- **Limite GitHub** : 60 interrogations par heure et par adresse IP pour l'API ; un poste en fait 6 par jour. Au-delà, la redirection `/releases/latest` prend le relais (elle exige que la dernière version publiée du dépôt soit une version OfficiaTech).
- **Signature de code** (recommandé à terme) : un certificat de signature Windows supprimerait l'avertissement SmartScreen à la première installation.

Si un autre logiciel est publié dans le même dépôt

Hospitalis partage le dépôt GOJO. Si ses versions sont un jour publiées dans Releases, utilisez des étiquettes distinctes (`hospitalis-v…`) : OfficiaTech les ignore. La source de secours ne regardant que la toute dernière version du dépôt, l'API GitHub reste alors la source principale.

11 Incidents et retour arrière

Propriétaire Développeur

Une version publiée par erreur ou défectueuse : on ne « revient » pas en arrière, on publie vite une version corrigée plus grande.

---

1. **Arrêter la diffusion** : sur GitHub › Releases, ouvrez la version défectueuse et passez-la en **Draft** (brouillon) ou supprimez-la. Les ordinateurs qui ne l'ont pas encore téléchargée ne la verront plus.
2. **Corriger** : soit annuler la modification dans le code, soit la réparer.
3. **Publier une version plus grande** (par exemple 1.13.1) : tous les ordinateurs, y compris ceux qui avaient installé la mauvaise, passent à la version corrigée.
4. **Poste bloqué** (l'application ne démarre plus) : réinstaller la dernière bonne version depuis Releases ; les données sont conservées. En dernier recours, restaurer une sauvegarde (Paramètres › Sauvegarde).

Symptôme	Piste
Aucun poste ne voit la nouvelle version	Version bien publiée (pas brouillon) ? Numéro plus grand ? Fichier `OfficiaTech-Setup-x.y.z.exe` présent ? Patienter jusqu'à 4 h ou cliquer sur « Rechercher ».
« Vérification momentanément impossible »	Limite GitHub atteinte (réseau partagé) : la source de secours prend le relais ; sinon nouvel essai plus tard.
« Fichier de mise à jour altéré »	Téléchargement corrompu (réseau, antivirus) : il est rejeté ; nouvel essai automatique.
Windows demande une autorisation à chaque mise à jour	OfficiaTech a été installé « pour tous les utilisateurs » : réinstaller en mode « pour moi seul ».
La compilation GitHub est rouge	Ouvrir l'exécution dans Actions : l'étape en échec indique le test ou l'erreur ; corriger puis renvoyer.

A Aide-mémoire

À imprimer.

Publier une version

- [ ] `package.json` : version augmentée (et `src/lib/about.js`).
- [ ] `CHANGELOG.md` : nouveautés en haut, en français.
- [ ] `npm test` : « fail 0 » ; `npm run build` : OK.
- [ ] Push sur **main**, compilation verte.
- [ ] Actions › « OfficiaTech — installateur Windows » › Run workflow › ☑ publier.
- [ ] Releases : **officia-vX.Y.Z** avec `OfficiaTech-Setup-X.Y.Z.exe`.
- [ ] Si `officia/server` a changé : relancer l'installation du serveur.
- [ ] Vérifier sur un poste : Paramètres › Mises à jour › Rechercher.

Commandes

Besoin	Commande
Tests	`cd officia && npm test`
Lancer en développement	`npm run dev`
Publier par étiquette	`git tag officia-vX.Y.Z && git push origin officia-vX.Y.Z`
Se connecter au VPS	`ssh -i C:\chemin\vers\cle root@IP_DU_VPS`
Mettre à jour le serveur	commande d'installation (chapitre 7)
État du serveur	`systemctl status officiatech`

Contact

Propriétaire et développeur	**M. TETCHI KOREY JOEL**, Comptable
Téléphone	+225 07 19 37 80 57 · +225 01 60 29 61 49
E-mail	heliontech97@gmail.com